THE UNIVERSITY OF ALABAMA®

WHERE LEGENDS ARE MADE®

Jiacheng Li, Ph.D.

B-Trees (Deletion)

CS 201: Data Structures and Algorithms

Jiacheng Li, Ph.D.

Deletion

Case 1: ***The search arrives at a leaf node*** x. If x contains key k, then delete k from x. If x does not contain key k, then k was not in the B-tree and nothing else needs to be done.

Case 2: ***The search arrives at an internal node*** x *that contains key* k. Let $k = x.\mathrm{key}_i$. One of the following three cases applies, depending on the number of keys in $x.c_i$ (the child of x that precedes k) and $x.c_{i+1}$ (the child of x that follows k).

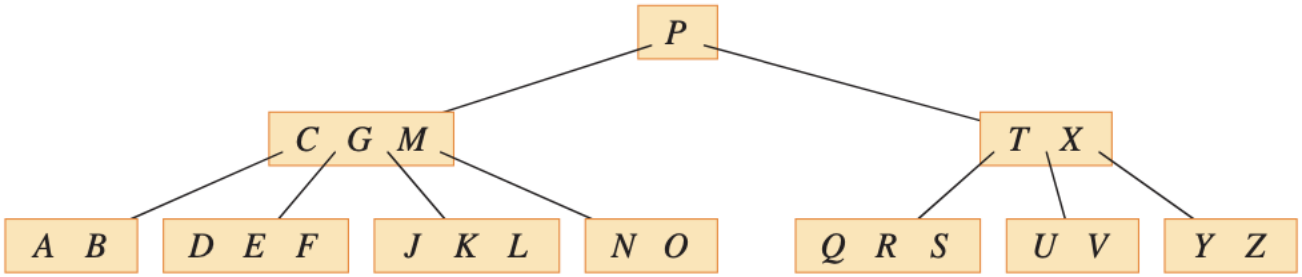
Case 3: ***The search arrives at an internal node*** x *that does not contain key* k. Continue searching down the tree while ensuring that each node visited has at least t keys. To do so, determine the root $x.c_i$ of the appropriate subtree that must contain k, if k is in the tree at all. If $x.c_i$ has only $t-1$ keys, execute case 3a or 3b as necessary to guarantee descending to a node containing at least t keys. Then finish by recursing on the appropriate child of x.

Deleting F from a B-tree

Case 1: *The search arrives at a leaf node x.* If x contains key k, then delete k from x. If x does not contain key k, then k was not in the B-tree and nothing else needs to be done.

Deleting a key from a B-tree

Deletion of F.

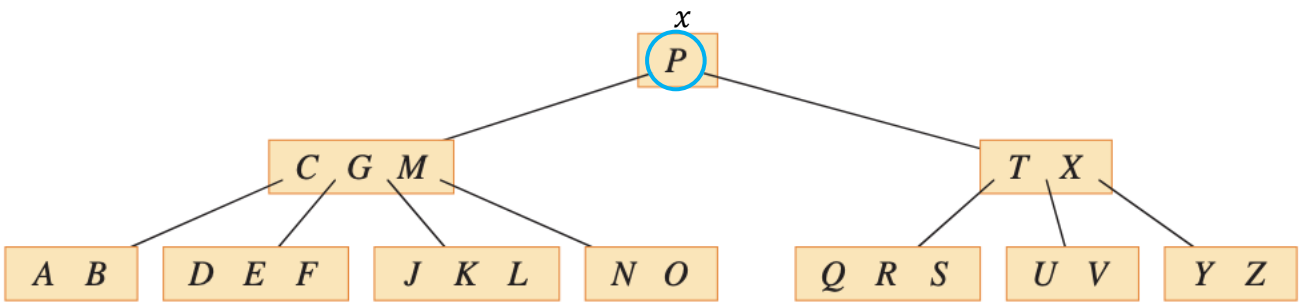


(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Deleting a key from a B-tree

Deletion of F.

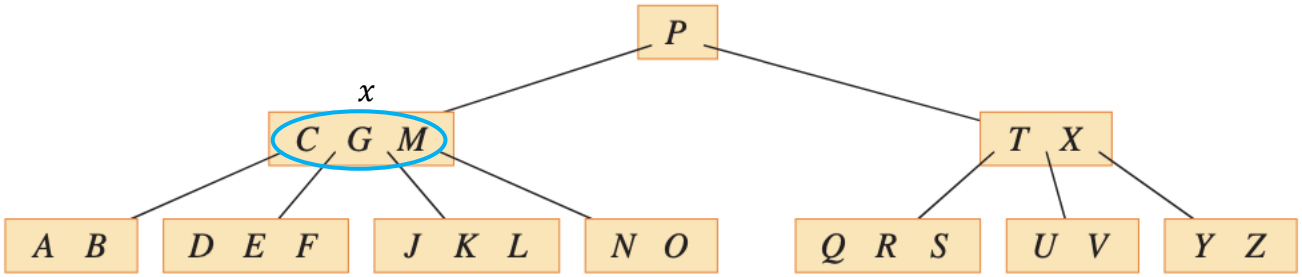


(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Deleting a key from a B-tree

Deletion of F.

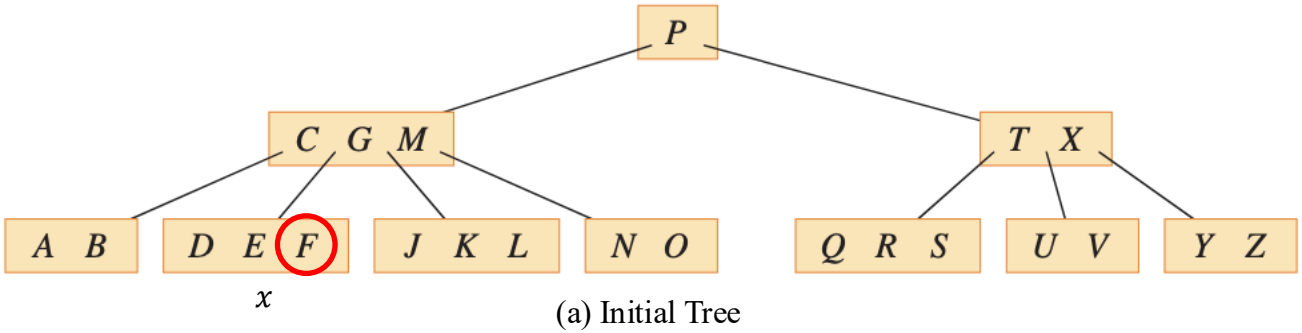


(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Deleting a key from a B-tree

Deletion of F.



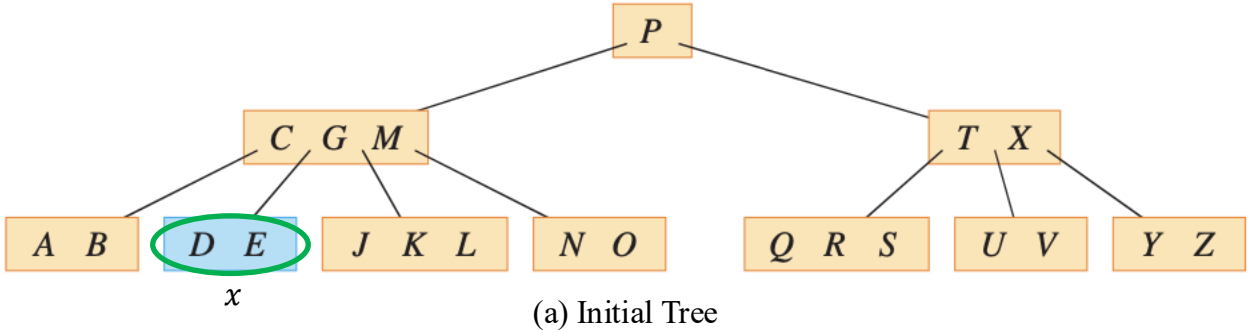
(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Case 1: *The search arrives at a leaf node* x. If x contains key k, then delete k from x. If x does not contain key k, then k was not in the B-tree and nothing else needs to be done.

Deleting a key from a B-tree

Deletion of F.



(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

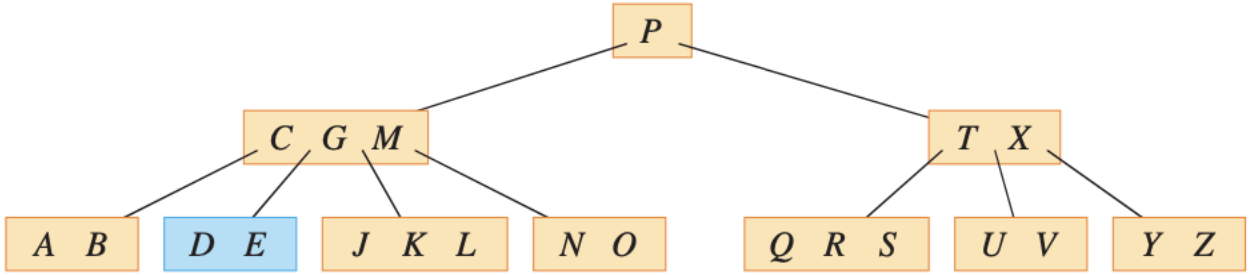
Case 1: *The search arrives at a leaf node* x. If x contains key k, then delete k from x. If x does not contain key k, then k was not in the B-tree and nothing else needs to be done.

Deleting M from a B-tree

Case 2: *The search arrives at an internal node x that contains key k.* Let $k = x.key_i$. One of the following three cases applies, depending on the number of keys in $x.c_i$ (the child of x that precedes k) and $x.c_{i+1}$ (the child of x that follows k).

Deleting a key from a B-tree

Deletion of M.

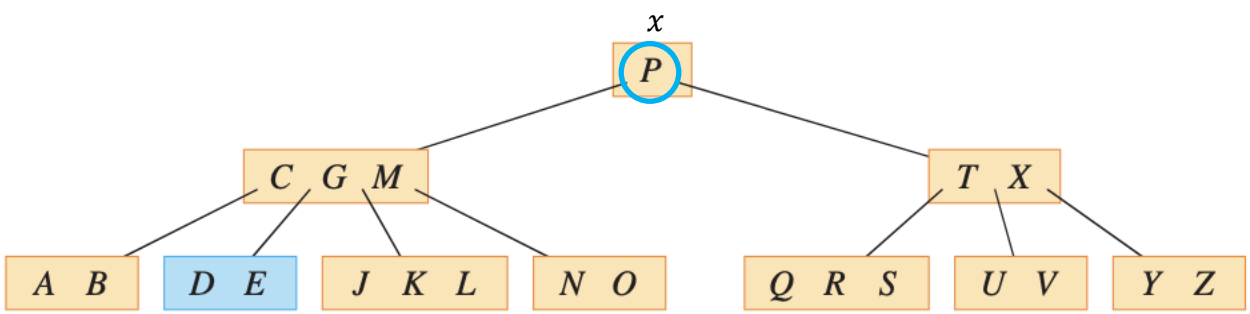


(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Deleting a key from a B-tree

Deletion of M.

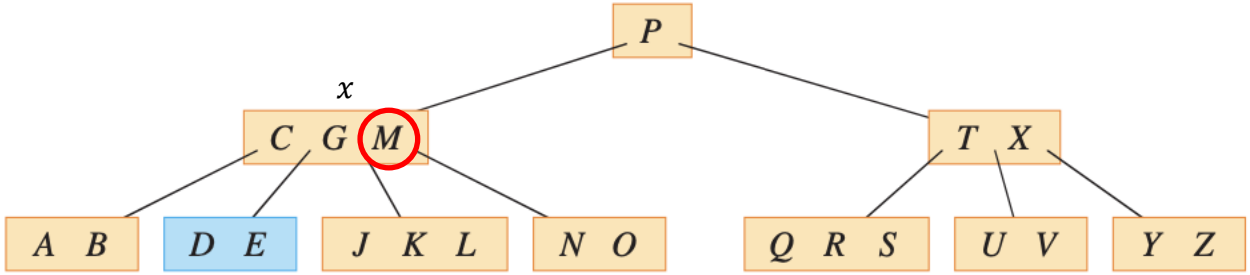


(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Deleting a key from a B-tree

Deletion of M.

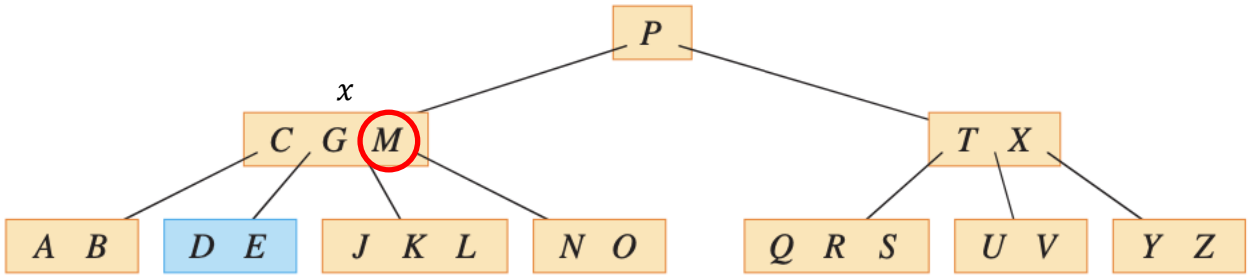


(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Deleting a key from a B-tree

Deletion of M.



(a) Initial Tree

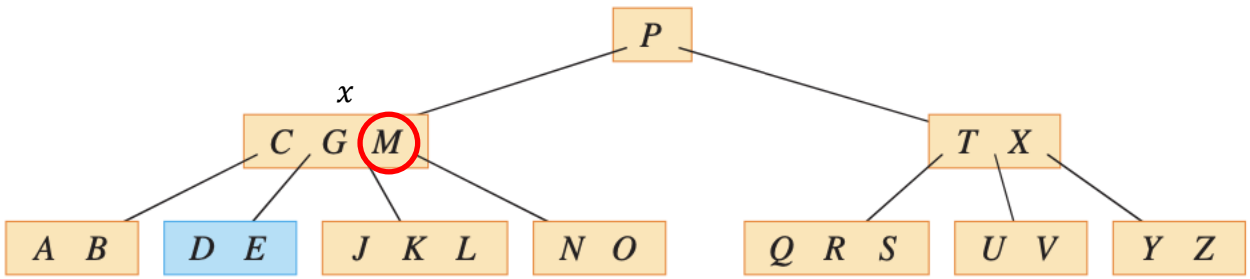
**The minimum degree for this B-tree is $t = 3$.

Case 2a: *$x.c_i$ has at least t keys.* Find the predecessor k' of k in the subtree rooted at $x.c_i$. Recursively delete k' from $x.c_i$, and replace k by k' in x. (Key k' can be found and deleted in a single downward pass.)

$x.c_3 = [J, K, L]$ $\quad$ $K = M$ $\quad$ $K' = L$

Deleting a key from a B-tree

Deletion of M.



(a) Initial Tree

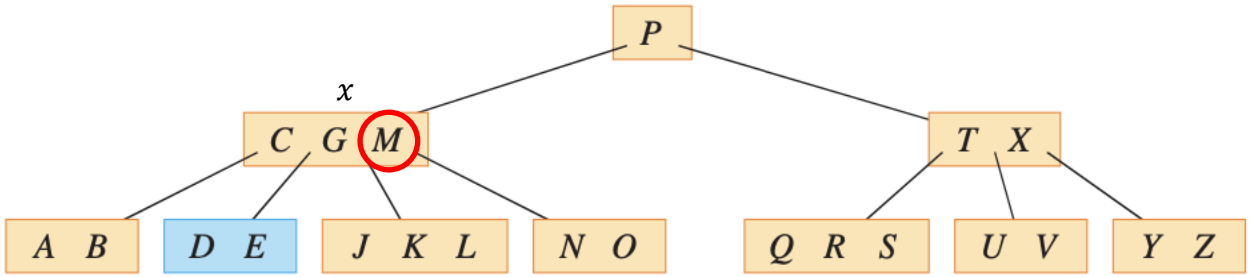
**The minimum degree for this B-tree is $t = 3$.

Case 2a: $x.c_i$ *has at least* t *keys*. Find the predecessor k' of k in the subtree rooted at $x.c_i$. Recursively delete k' from $x.c_i$, and replace k by k' in x. (Key k' can be found and deleted in a single downward pass.)

If "L" was a leaf node key, it would be simple: just remove it.

Deleting a key from a B-tree

Deletion of M.



(a) Initial Tree

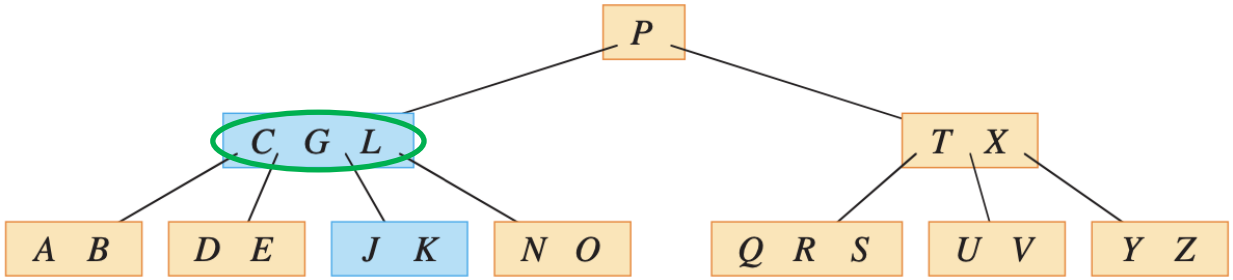
**The minimum degree for this B-tree is $t = 3$.

Case 2a: $x.c_i$ *has at least* t *keys*. Find the predecessor k' of k in the subtree rooted at $x.c_i$. Recursively delete k' from $x.c_i$, and replace k by k' in x. (Key k' can be found and deleted in a single downward pass.)

If "*L*" had children (say if it was not in a leaf node), the recursion would handle the redistribution or merging of these children to maintain minimum degree requirements.

Deleting a key from a B-tree

Deletion of M.



(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

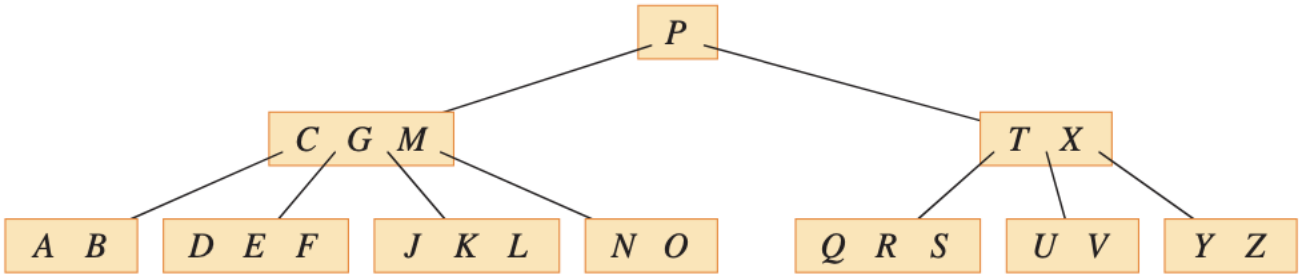
Case 2a: *$x.c_i$ has at least t keys.* Find the predecessor k' of k in the subtree rooted at $x.c_i$. Recursively delete k' from $x.c_i$, and replace k by k' in x. (Key k' can be found and deleted in a single downward pass.)

Deleting a key from a B-tree

Case 2b: *$x.c_i$ has $t-1$ keys and $x.c_{i+1}$ has at least t keys.* This case is symmetric to case 2a. Find the successor k' of k in the subtree rooted at $x.c_{i+1}$. Recursively delete k' from $x.c_{i+1}$, and replace k by k' in x. (Again, finding and deleting k' can be done in a single downward pass.)

Deleting a key from a B-tree

Deletion of C.

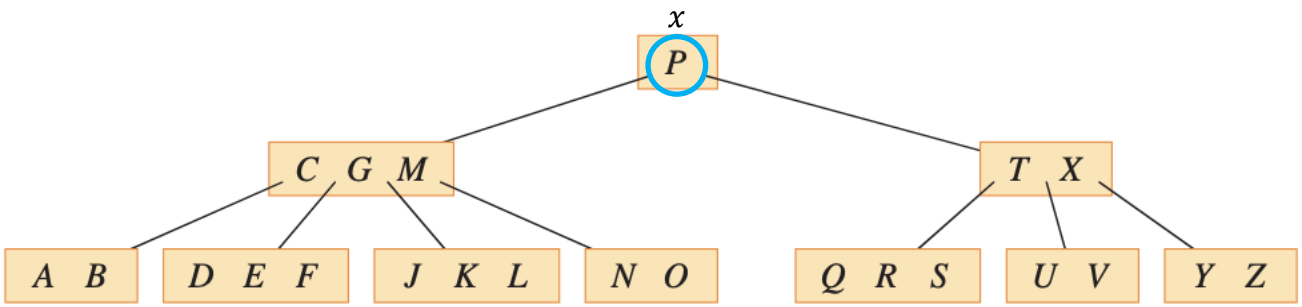


(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Deleting a key from a B-tree

Deletion of C.

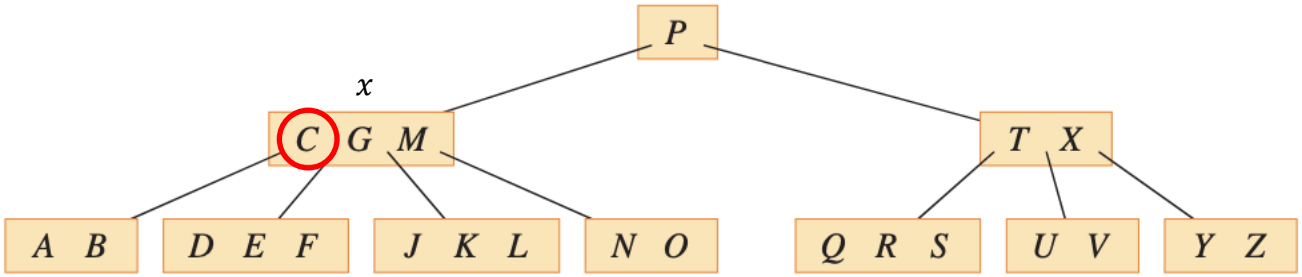


(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Deleting a key from a B-tree

Deletion of C.

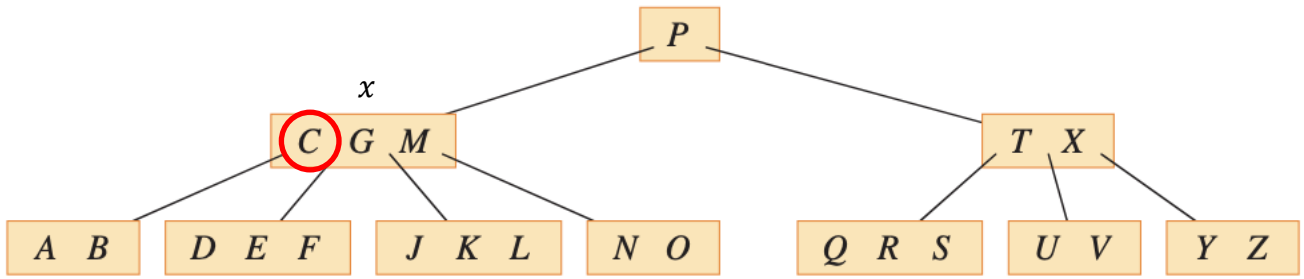


(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Deleting a key from a B-tree

Deletion of C.



(a) Initial Tree

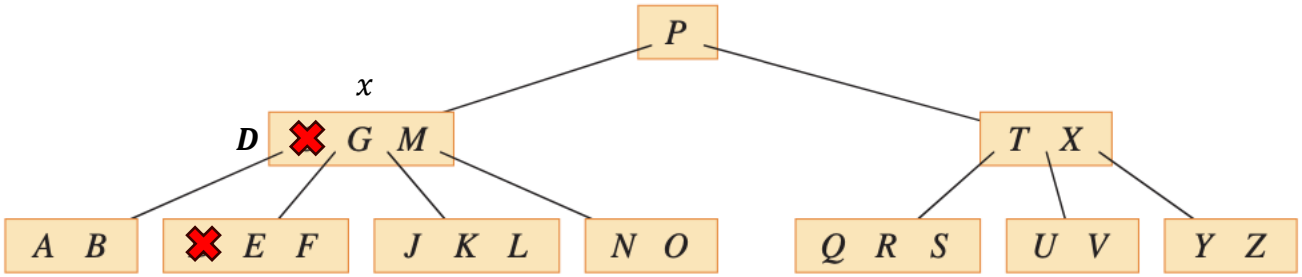
**The minimum degree for this B-tree is $t = 3$.

Case 2b: *$x.c_i$ has $t - 1$ keys and $x.c_{i+1}$ has at least t keys.* This case is symmetric to case 2a. Find the successor k' of k in the subtree rooted at $x.c_{i+1}$. Recursively delete k' from $x.c_{i+1}$, and replace k by k' in x. (Again, finding and deleting k' can be done in a single downward pass.)

$x.c_i = [A, B]$ $\quad$ $x.c_{i+1} = [D, E, F]$ $\quad$ $K' = D$ $\quad$ $K = C$

Deleting a key from a B-tree

Deletion of C.



(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Case 2b: *$x.c_i$ has $t-1$ keys and $x.c_{i+1}$ has at least t keys.* This case is symmetric to case 2a. Find the successor k' of k in the subtree rooted at $x.c_{i+1}$. Recursively delete k' from $x.c_{i+1}$, and replace k by k' in x. (Again, finding and deleting k' can be done in a single downward pass.)

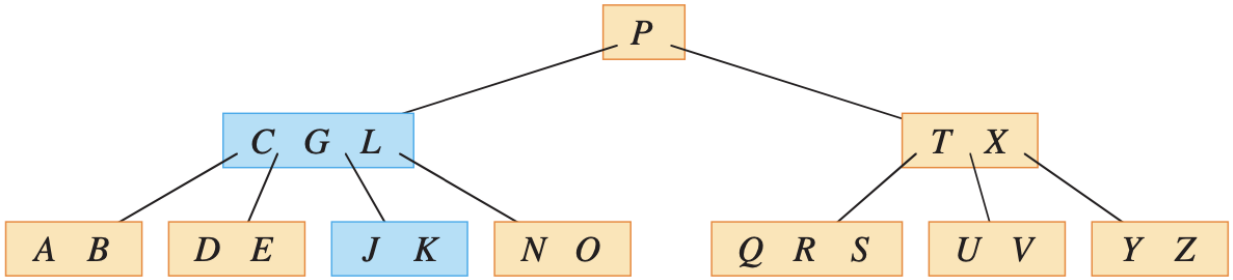
$x.c_i = [A, B]$ $\quad$ $x.c_{i+1} = [D, E, F]$ $\quad$ $K' = D$ $\quad$ $K = C$

Deleting G from a B-tree

Case 2: *The search arrives at an internal node x that contains key k.* Let $k = x.key_i$. One of the following three cases applies, depending on the number of keys in $x.c_i$ (the child of x that precedes k) and $x.c_{i+1}$ (the child of x that follows k).

Deleting a key from a B-tree

Deletion of G.

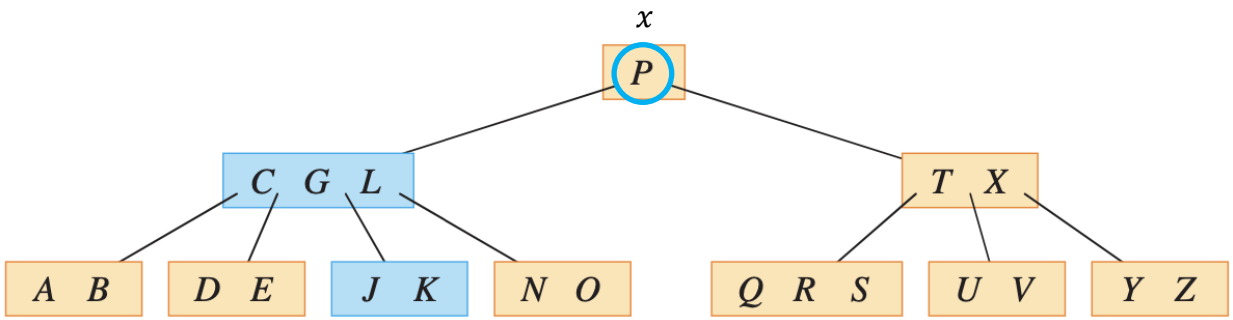


(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Deleting a key from a B-tree

Deletion of G.

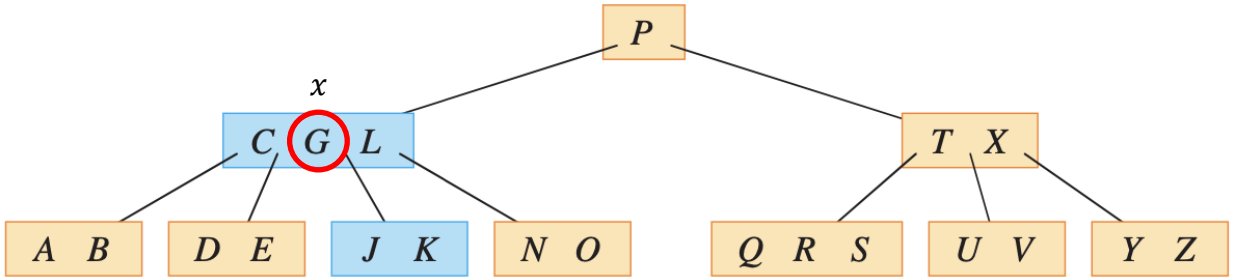


(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Deleting a key from a B-tree

Deletion of G.

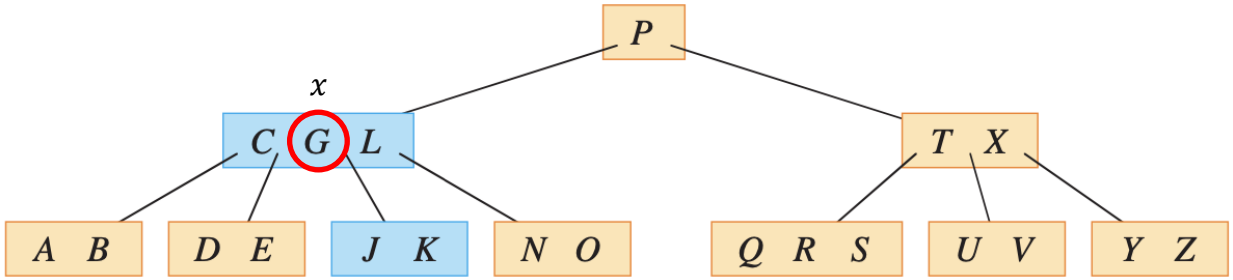


(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Deleting a key from a B-tree

Deletion of G.



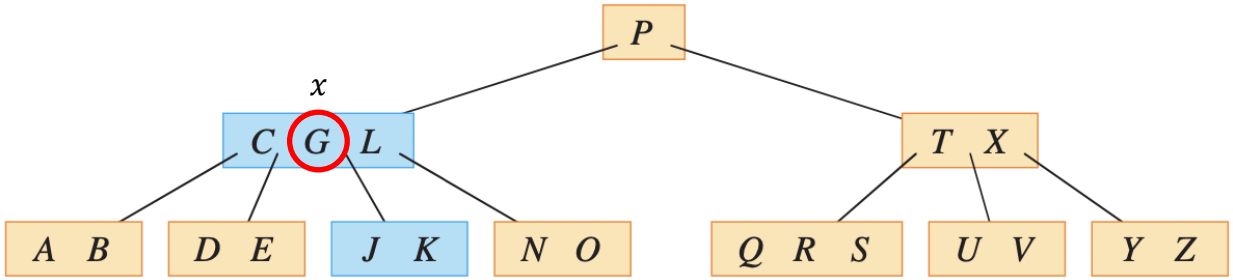
(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Case 2c: *Both $x.c_i$ and $x.c_{i+1}$ have $t-1$ keys.* Merge k and all of $x.c_{i+1}$ into $x.c_i$, so that x loses both k and the pointer to $x.c_{i+1}$, and $x.c_i$ now contains $2t-1$ keys. Then free $x.c_{i+1}$ and recursively delete k from $x.c_i$.

Deleting a key from a B-tree

Deletion of G.



(a) Initial Tree

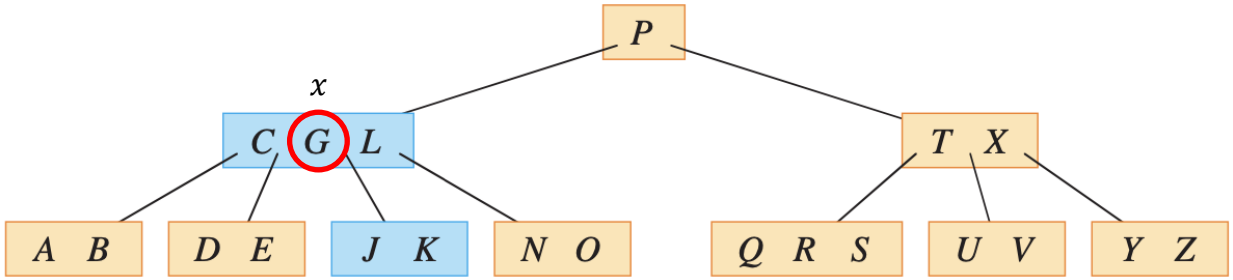
**The minimum degree for this B-tree is $t = 3$.

Case 2c: *Both $x.c_i$ and $x.c_{i+1}$ have $t-1$ keys.* Merge k and all of $x.c_{i+1}$ into $x.c_i$, so that x loses both k and the pointer to $x.c_{i+1}$, and $x.c_i$ now contains $2t-1$ keys. Then free $x.c_{i+1}$ and recursively delete k from $x.c_i$.

$x.c_i = [D, E]$ $\quad$ $x.c_{i+1} = [J, K]$ $\quad$ $k = G$

Deleting a key from a B-tree

Deletion of G.



$x.c_i = [D, E, G, J, K]$ (a) Initial Tree

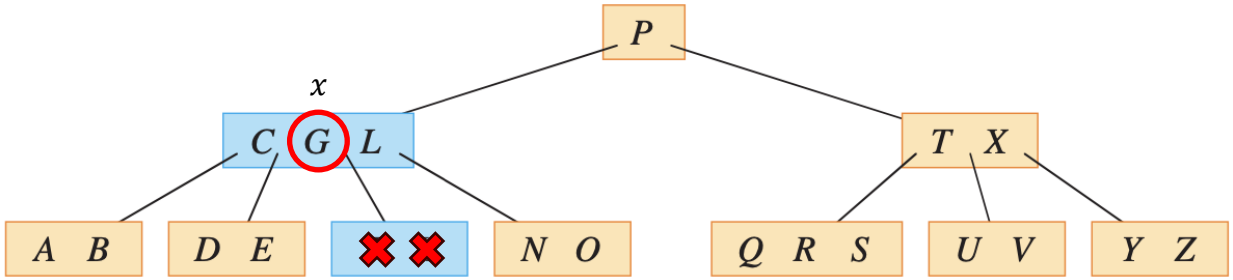
**The minimum degree for this B-tree is $t = 3$.

Case 2c: *Both $x.c_i$ and $x.c_{i+1}$ have $t - 1$ keys.* Merge k and all of $x.c_{i+1}$ into $x.c_i$, so that x loses both k and the pointer to $x.c_{i+1}$, and $x.c_i$ now contains $2t - 1$ keys. Then free $x.c_{i+1}$ and recursively delete k from $x.c_i$.

$x.c_i = [D, E]$ $\quad$ $x.c_{i+1} = [J, K]$ $\quad$ $k = G$

Deleting a key from a B-tree

Deletion of G.



$x.c_i = [D, E, \cancel{G}, J, K]$

(a) Initial Tree

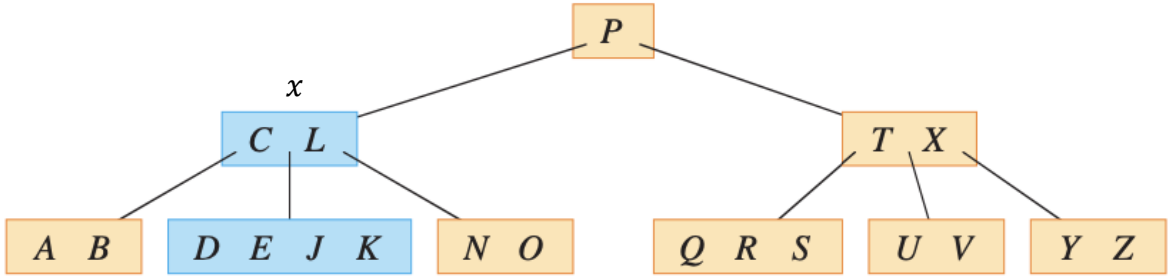
**The minimum degree for this B-tree is $t = 3$.

Case 2c: *Both $x.c_i$ and $x.c_{i+1}$ have $t-1$ keys.* Merge k and all of $x.c_{i+1}$ into $x.c_i$, so that x loses both k and the pointer to $x.c_{i+1}$, and $x.c_i$ now contains $2t-1$ keys. Then free $x.c_{i+1}$ and recursively delete k from $x.c_i$.

$x.c_i = [D, E]$ $\quad$ $x.c_{i+1} = [J, K]$ $\quad$ $k = G$

Deleting a key from a B-tree

Deletion of G.



(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Case 2c: *Both $x.c_i$ and $x.c_{i+1}$ have $t-1$ keys.* Merge k and all of $x.c_{i+1}$ into $x.c_i$, so that x loses both k and the pointer to $x.c_{i+1}$, and $x.c_i$ now contains $2t-1$ keys. Then free $x.c_{i+1}$ and recursively delete k from $x.c_i$.

Case 2

Case 2: ***The search arrives at an internal node*** x *that contains key* k. Let $k = x.\text{key}_i$. One of the following three cases applies, depending on the number of keys in $x.c_i$ (the child of x that precedes k) and $x.c_{i+1}$ (the child of x that follows k).

Case 2a: $x.c_i$ *has at least* t *keys*. Find the predecessor k' of k in the subtree rooted at $x.c_i$. Recursively delete k' from $x.c_i$, and replace k by k' in x. (Key k' can be found and deleted in a single downward pass.)

Case 2b: $x.c_i$ *has* $t-1$ *keys and* $x.c_{i+1}$ *has at least* t *keys*. This case is symmetric to case 2a. Find the successor k' of k in the subtree rooted at $x.c_{i+1}$. Recursively delete k' from $x.c_{i+1}$, and replace k by k' in x. (Again, finding and deleting k' can be done in a single downward pass.)

Case 2c: *Both* $x.c_i$ *and* $x.c_{i+1}$ *have* $t-1$ *keys*. Merge k and all of $x.c_{i+1}$ into $x.c_i$, so that x loses both k and the pointer to $x.c_{i+1}$, and $x.c_i$ now contains $2t-1$ keys. Then free $x.c_{i+1}$ and recursively delete k from $x.c_i$.

Deleting D from a B-tree

Case 3: ***The search arrives at an internal node*** x *that does not contain key* k. Continue searching down the tree while ensuring that each node visited has at least t keys. To do so, determine the root $x.c_i$ of the appropriate subtree that must contain k, if k is in the tree at all. If $x.c_i$ has only $t-1$ keys, execute case 3a or 3b as necessary to guarantee descending to a node containing at least t keys. Then finish by recursing on the appropriate child of x.

**Before deleting a key from B-tree, we first ensures that the node you're about to work on is safe — meaning it has at least t keys, not the minimum t−1.

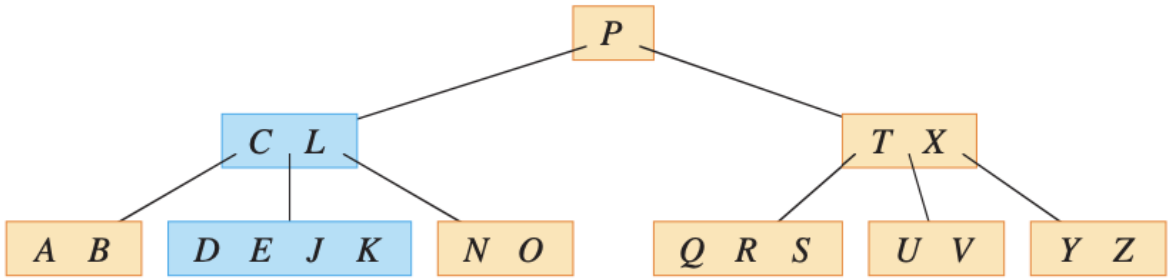
✓ When descending to a child that has only the minimum number of keys, we must fix it before descending.

Deleting a key from a B-tree

**Before deleting a key from B-tree, we first ensures that the node you're about to work on is safe — meaning it has at least t keys, not the minimum t−1.

✓ When descending to a child that has only the minimum number of keys, we must fix it before descending.

Deletion of D.



(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

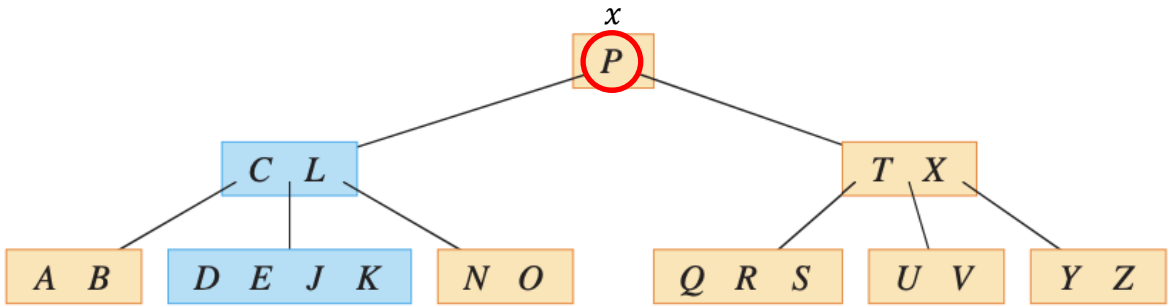
Case 3b: *$x.c_i$ and each of $x.c_i$'s immediate siblings have $t - 1$ keys.* (It is possible for $x.c_i$ to have either one or two siblings.) Merge $x.c_i$ with one sibling, which involves moving a key from x down into the new merged node to become the median key for that node.

Deleting a key from a B-tree

**Before deleting a key from B-tree, we first ensures that the node you're about to work on is safe — meaning it has at least t keys, not the minimum t−1.

✓ When descending to a child that has only the minimum number of keys, we must fix it before descending.

Deletion of D.



(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

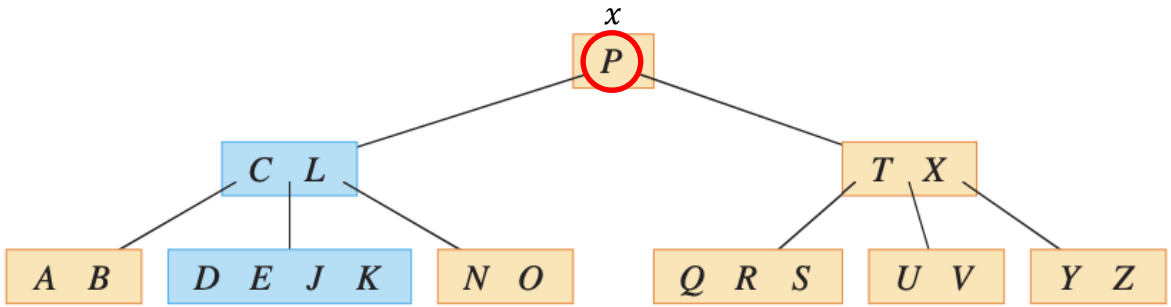
Case 3b: *$x.c_i$ and each of $x.c_i$'s immediate siblings have $t - 1$ keys.* (It is possible for $x.c_i$ to have either one or two siblings.) Merge $x.c_i$ with one sibling, which involves moving a key from x down into the new merged node to become the median key for that node.

Deleting a key from a B-tree

**Before deleting a key from B-tree, we first ensures that the node you're about to work on is safe — meaning it has at least t keys, not the minimum t−1.

✓ When descending to a child that has only the minimum number of keys, we must fix it before descending.

Deletion of D.



(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Case 3b: *$x.c_i$ and each of $x.c_i$'s immediate siblings have $t - 1$ keys.* (It is possible for $x.c_i$ to have either one or two siblings.) Merge $x.c_i$ with one sibling, which involves moving a key from x down into the new merged node to become the median key for that node.

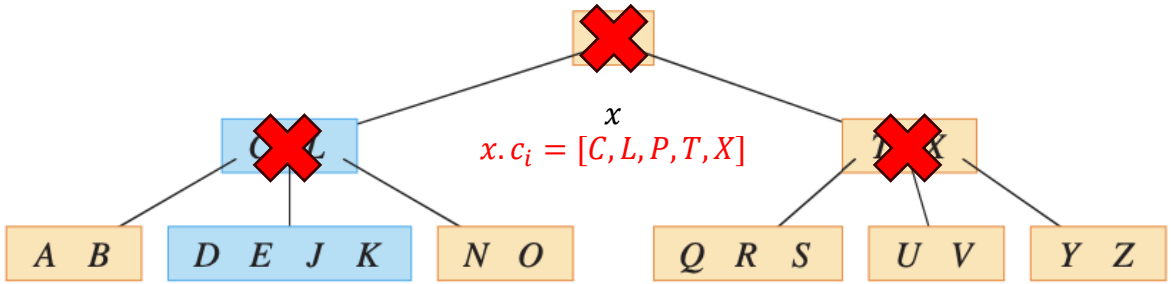
$x.c_i = [C, L]$ $\quad$ $x.c_{i+1} = [T, X]$ $\quad$ $x.c_i = [C, L, P, T, X]$

Deleting a key from a B-tree

**Before deleting a key from B-tree, we first ensures that the node you're about to work on is safe — meaning it has at least t keys, not the minimum t−1.

✓ When descending to a child that has only the minimum number of keys, we must fix it before descending.

Deletion of D.



(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

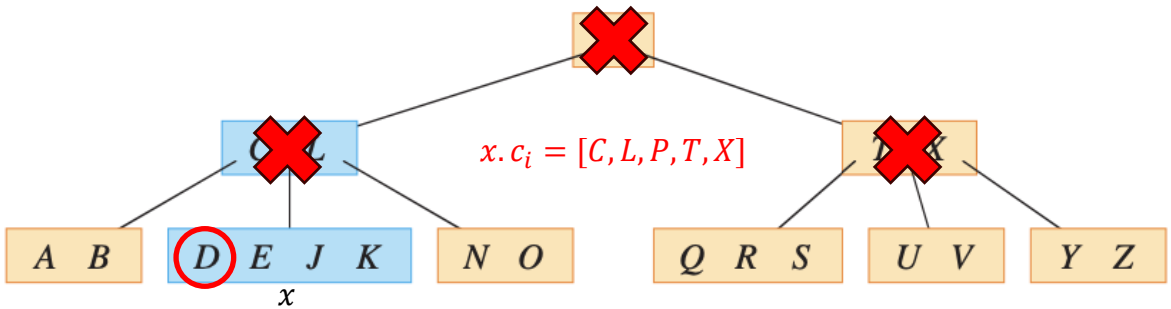
Case 3b: *$x.c_i$ and each of $x.c_i$'s immediate siblings have $t - 1$ keys.* (It is possible for $x.c_i$ to have either one or two siblings.) Merge $x.c_i$ with one sibling, which involves moving a key from x down into the new merged node to become the median key for that node.

Deleting a key from a B-tree

**Before deleting a key from B-tree, we first ensures that the node you're about to work on is safe — meaning it has at least t keys, not the minimum t−1.

✓ When descending to a child that has only the minimum number of keys, we must fix it before descending.

Deletion of D.



(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

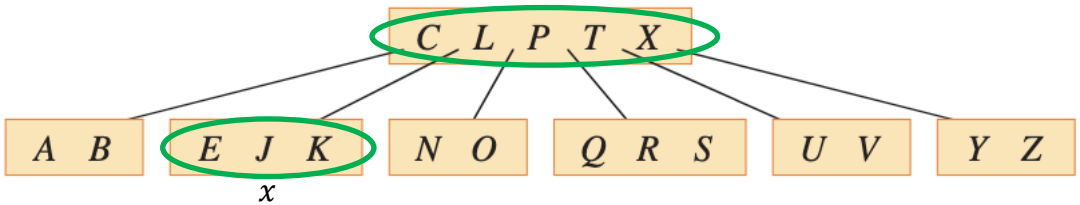
Case 1: *The search arrives at a leaf node* x. If x contains key k, then delete k from x. If x does not contain key k, then k was not in the B-tree and nothing else needs to be done.

Deleting a key from a B-tree

**Before deleting a key from B-tree, we first ensures that the node you're about to work on is safe — meaning it has at least t keys, not the minimum t−1.

✓ When descending to a child that has only the minimum number of keys, we must fix it before descending.

Deletion of D.



(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

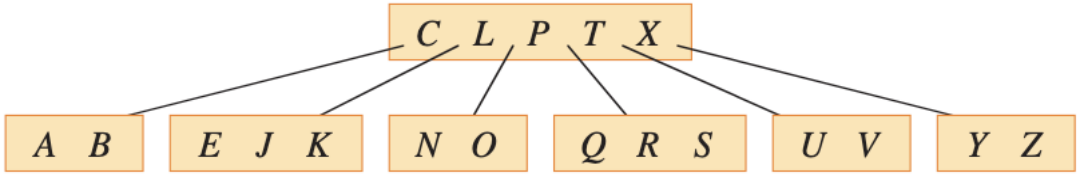
Case 1: *The search arrives at a leaf node* x. If x contains key k, then delete k from x. If x does not contain key k, then k was not in the B-tree and nothing else needs to be done.

Deleting B from a B-tree

Case 3a: *$x.c_i$ has only $t-1$ keys but has an immediate sibling with at least t keys*. Give $x.c_i$ an extra key by moving a key from x down into $x.c_i$, moving a key from $x.c_i$'s immediate left or right sibling up into x, and moving the appropriate child pointer from the sibling into $x.c_i$.

Deleting a key from a B-tree

Deletion of B.

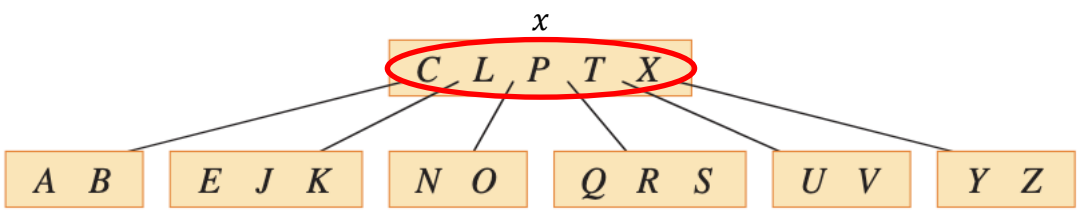


(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Deleting a key from a B-tree

Deletion of B.



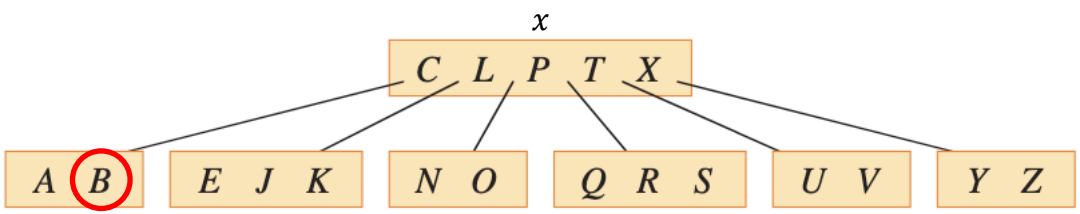
(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Case 3a: *$x.c_i$ has only $t - 1$ keys but has an immediate sibling with at least t keys*. Give $x.c_i$ an extra key by moving a key from x down into $x.c_i$, moving a key from $x.c_i$'s immediate left or right sibling up into x, and moving the appropriate child pointer from the sibling into $x.c_i$.

Deleting a key from a B-tree

Deletion of B.



(a) Initial Tree

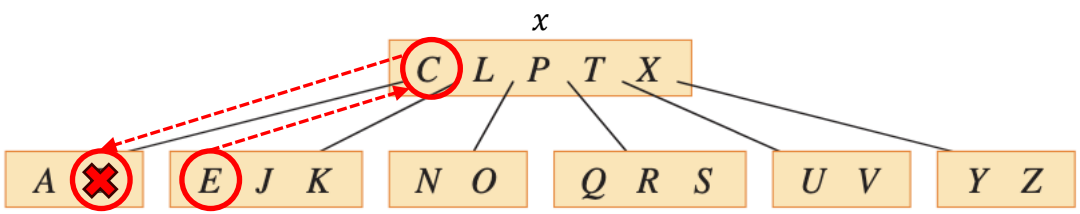
**The minimum degree for this B-tree is $t = 3$.

Case 3a: *$x.c_i$ has only $t - 1$ keys but has an immediate sibling with at least t keys*. Give $x.c_i$ an extra key by moving a key from x down into $x.c_i$, moving a key from $x.c_i$'s immediate left or right sibling up into x, and moving the appropriate child pointer from the sibling into $x.c_i$.

$$x.c_i = [A, B] \qquad x.c_{i+1} = [E, J, K]$$

Deleting a key from a B-tree

Deletion of B.



(a) Initial Tree

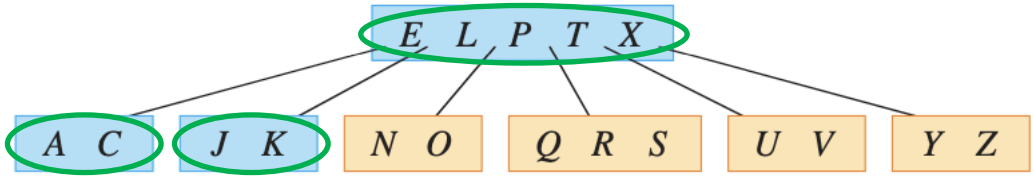
**The minimum degree for this B-tree is $t = 3$.

Case 3a: *$x.c_i$ has only $t - 1$ keys but has an immediate sibling with at least t keys*. Give $x.c_i$ an extra key by moving a key from x down into $x.c_i$, moving a key from $x.c_i$'s immediate left or right sibling up into x, and moving the appropriate child pointer from the sibling into $x.c_i$.

$x.c_i = [A, C]$ $\quad$ $x.c_{i+1} = [J, K]$

Deleting a key from a B-tree

Deletion of B.



(a) Initial Tree

**The minimum degree for this B-tree is $t = 3$.

Case 3a: *$x.c_i$ has only $t - 1$ keys but has an immediate sibling with at least t keys*. Give $x.c_i$ an extra key by moving a key from x down into $x.c_i$, moving a key from $x.c_i$'s immediate left or right sibling up into x, and moving the appropriate child pointer from the sibling into $x.c_i$.

Questions?

ROLL TIDE®